IT079IU - Principles of Database Management - Lab 6

Java + MySQL: Connect, Query & Stay Safe

Le Thanh Trung - ITCSIU25055 - Lab 6

November 2026

Course IT079IU — Principles of Database Management
Lab 6 of 10
Student Le Thanh Trung · **Student ID** ITCSIU25055
Database EduConnect — `educonnect` on MySQL 8.4 (port 33306)
JDK Temurin OpenJDK 25.0.2 · **Driver** MySQL Connector/J 8.4.0 · **Build** Maven 3.10

Every demonstration in this report was executed against the running EduConnect database. Captured output is in `evidence/`, the Java source is in `project/src/main/java/educonnect/`, and the supporting SQL is in `src/`.

Contents

- **1** Objective
- **2** Environment and database setup
- **3** Exercise 1 — Fix the broken code
  - 3.1 Task 1.1 — Issues in `BrokenQueryRunner`
  - 3.2 Task 1.2 — `SafeQueryRunner`
  - 3.3 Task 1.3 — Connection leak demonstration
- **4** Exercise 2 — Query runner application
  - 4.1 Task 2.1 — Swing UI and `SwingWorker`
  - 4.2 Task 2.2 — Parameterised search with `LIKE`
  - 4.3 Task 2.3 — `ResultSet` after connection close
- **5** Exercise 3 — Security and edge cases
  - 5.1 Task 3.1 — Full SQL injection demonstration
  - 5.2 Task 3.2 — Calling `EnrollStudent` via `CallableStatement`
  - 5.3 Task 3.3 — `GetCourseReport` and multiple result sets
- **6** Part 8 — Thinking questions
- **7** Part 7 checklist
- **8** Reflection
- **Appendix A** Contents of the submission bundle

---

1 1. Objective

This lab connects the EduConnect database to a real Java application through JDBC. The work covers the full lifecycle: opening a connection, running queries safely, reading a `ResultSet`, and closing resources in the right order. The emphasis is on what goes wrong in production: SQL injection, connection leaks, `ResultSet` misuse, and a frozen Swing UI. Three exercises move from fixing a deliberately broken class, through a Swing query runner, to security and edge-case demonstrations.

---

2 2. Environment and database setup

The database and seed data are reused from Lab 4 (8 users, 5 courses, 11 enrolments, 5 lessons, 9 attendance rows). Lab 6 needs three objects that did not exist, so `src/00_db_objects.sql` creates them:

- `StudentProgressReport` view — per-student enrolment count and average grade.
- `EnrollStudent(userID, courseID, OUT message)` — validates and inserts an enrolment.
- `GetCourseReport(courseID)` — returns `FullName`, `EnrollDate`, `GradeDisplay`.

MySQL runs in Docker on host port 33306 with credentials `root` / `root`. The connection URL used throughout is:

```
jdbc:mysql://localhost:33306/educonnect?useSSL=false&allowPublicKeyRetrieval=true&serve
```

`allowPublicKeyRetrieval=true` is required because MySQL 8.4 uses `caching_sha2_password` and the lab connection is not over TLS; it does not change the security model of the lab, only allows the initial key exchange.

---

3 3. Exercise 1 — Fix the broken code

3.1 3.1 Task 1.1 — Issues in `BrokenQueryRunner`

`BrokenQueryRunner` (Part 4.3 of the handout) contains at least six distinct issues:

#	Issue	Why it matters in production	Lines
1	SQL injection via string concatenation (`' + nameInput + '`)	A malicious input such as `' OR '1'='1` returns every user; `'; DROP TABLE Users; --` destroys data	6-7
2	`Connection`, `Statement`, `ResultSet` never closed	Each call leaks a socket and a server-side cursor; 151 leaks exhaust MySQL	9-11
3	`Statement` instead of `PreparedStatement`	No parameter binding, so injection is possible and the plan cannot be reused	10
4	Column access by index (`getInt(1)`, `getString(2)`, `getString(3)`)	Breaks silently if the `SELECT` column order changes	13-15
5	No `rs.next()` check before reading	Throws `SQLException` ("ResultSet not positioned on a row") on empty result	13
6	`catch (SQLException e) { e.printStackTrace(); }`	Stack trace leaks to stdout/stderr; the caller never learns the query failed	17-18
7	`Connection` created outside try-with-resources	Even on exception the connection stays open	9

3.2 3.2 Task 1.2 — `SafeQueryRunner`

Rewritten version using `PreparedStatement`, try-with-resources, column names, and a `List<String[]>` return:

```java
public static List<String[]> searchStudentByName(String nameInput) {
    List<String[]> results = new ArrayList<>();
    String sql = "SELECT UserID, FullName, Email FROM Users " +
                 "WHERE Role = 'student' AND FullName = ?";
    try (
        Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)
    ) {
        ps.setString(1, nameInput);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                results.add(new String[] {
                    String.valueOf(rs.getInt("UserID")),
                    rs.getString("FullName"),
                    rs.getString("Email")
                });
            }
        }
    } catch (SQLException e) {
        throw new RuntimeException("Query failed: " + e.getMessage(), e);
    }
    return results;
}
```

Test results (`evidence/safe_query.txt`):

```
normal size: 1
inject size: 0
1|Nguyen Van An|an.nguyen@edu.vn
```

A normal input returns one row; the injection attempt returns zero rows because the payload is treated as a literal string to match, not as SQL.

Why `PreparedStatement` prevents the injection (protocol level): MySQL receives two separate messages. First, the SQL template `SELECT ... WHERE FullName = ?` is sent and parsed/prepared by the server. Second, the parameter value is sent as a typed data value in the binary protocol. The server never substitutes the value back into a SQL string, so the value is never re-parsed as SQL. Even `' OR '1'='1` arrives as a single VARCHAR value and is compared against `FullName` literally.

3.3 3.3 Task 1.3 — Connection leak demonstration

A loop opened connections without closing them (`ConnectionLeakTest`). MySQL rejected the 153rd connection:

```
Opened connection 152
Failed at connection 153: Data source rejected establishment of connection,
  message from server: "Too many connections"
SQLState: 08004 ErrorCode: 1040
```

(`evidence/connection_leak.txt`.)

The Java exception is `java.sql.SQLNonTransientConnectionException` with error code 1040 ("Too many connections"). MySQL's default `max_connections` is 151; the extra headroom comes from the one reserved administrative connection.

Math for a real web application: at one leaked connection per request and 1,000 requests/day, the default limit of 151 is exhausted after 151 requests, i.e. on the first day (151 / 1000 = 0.151 days ≈ 3.6 hours). Even at 100 requests/day it takes 1.51 days. The fix is try-with-resources on every `Connection`, `Statement`, and `ResultSet`.

---

4 4. Exercise 2 — Query runner application

4.1 4.1 Task 2.1 — Swing UI and `SwingWorker`

`MainFrame` builds the UI from Part 4.5.2: a SQL text field, an Execute button, a `JTable` in a `JScrollPane`, and a status label. The query runs inside a `SwingWorker`:

```
SwingWorker<Void, Void> worker = new SwingWorker<>() {
    @Override
    protected Void doInBackground() throws Exception {
        // DB work here, on a background thread
        ...
    }
    @Override
    protected void done() {
        btnExecute.setEnabled(true);
        lblStatus.setText("Done");
    }
};
worker.execute();
```

The Execute button is disabled before `execute()` and re-enabled in `done()`, so a slow query cannot be double-submitted.

Why `SwingWorker` is necessary: Swing has a single Event Dispatch Thread (EDT) that repaints the window and processes clicks. Running `DBConnection.getConnection()` on the EDT blocks repaints, so the window freezes and appears crashed. `doInBackground()` runs on a worker thread; `done()` runs back on the EDT and is the only place that may touch Swing components. Temporarily removing `SwingWorker` and running `SELECT SLEEP(3)` freezes the window for three seconds and proves the point.

4.2 4.2 Task 2.2 — Parameterised search with `LIKE`

A separate panel searches `StudentProgressReport` by partial name. The pattern is built in Java and bound as a parameter:

```
String pattern = "%" + nameInput + "%";
ps.setString(1, pattern);   // safe: pattern is the parameter value, not SQL
```

SQL: `WHERE StudentName LIKE ?`.

Is this safe from SQL injection? Yes. The `?` placeholder is bound as a data value; MySQL receives the template `... LIKE ?` and the value `%Van%` as two separate messages. Building the `%` in Java does not create a vulnerability because the `%` characters become part of the parameter value, not of the SQL text. The only thing that would be unsafe is concatenating `nameInput` (or the `%`) into the SQL string.

Sample result (`evidence/student_search.txt`):

```
Nguyen Van An|3|7.75
Le Van Cuong|3|5.0
Hoang Van Em|2|6.0
```

4.3 4.3 Task 2.3 — `ResultSet` after connection close

`StudentDAO.getBadResultSet()` closes the connection before returning the `ResultSet`. Using it throws:

```
SQLException: Operation not allowed after ResultSet closed
SQLState: S1000 ErrorCode: 0
```

(`evidence/bad_resultset.txt`.)

Why closing `conn` invalidates `rs`: a `ResultSet` is a cursor backed by the underlying `Statement` and `Connection` (socket + server-side cursor). Closing the `Connection` closes the socket, which closes the `Statement`, which closes the `ResultSet`. Any later `rs.next()` or `rs.getString()` fails.

Returning a `ResultSet` from a method is a design mistake because the caller must keep the `Connection` open to use it, which leaks the connection lifecycle into the caller. The correct pattern is to process the `ResultSet` inside the method and return a `List<String[]>` (or a domain object). This is the Data Access Object (DAO) pattern.

Trade-offs: returning the full list materialises all rows in RAM, which is simple and lets the caller use the data freely, but does not scale to millions of rows. Streaming with `ps.setFetchSize(Integer.MIN_VALUE)` in MySQL Connector/J keeps one row in memory at a time but requires the `ResultSet` to stay open for the whole iteration and blocks the connection until consumption finishes.

---

5 5. Exercise 3 — Security and edge cases

5.1 5.1 Task 3.1 — Full SQL injection demonstration

A temporary `vulnerableSearch` using `Statement` and concatenation was run with four inputs. Results (`evidence/injection_demo.txt`):

```
=== Results for input: Nguyen Van An ===
1 | Nguyen Van An
=== End ===
=== Results for input: ' OR '1'='1' --  ===
1 | Nguyen Van An
2 | Tran Thi Bich
3 | Le Van Cuong
4 | Pham Thi Dung
5 | Hoang Van Em
6 | Dr. Minh Tuan
7 | Dr. Lan Anh
8 | Dr. Thanh Hung
=== End ===
=== Results for input: ' OR UserID > 3 --  ===
4 | Pham Thi Dung
5 | Hoang Van Em
6 | Dr. Minh Tuan
7 | Dr. Lan Anh
8 | Dr. Thanh Hung
=== End ===
=== Results for input: Nguyen' --  ===
=== End ===
```

The first payload closes the string literal (`'`), injects an always-true condition (`OR '1'='1'`), and comments out the rest (`--`). MySQL receives:

```
SELECT UserID, FullName, Role FROM Users WHERE FullName = '' OR '1'='1' -- '
```

With `PreparedStatement` and the same input, MySQL receives the template

```
SELECT UserID, FullName, Role FROM Users WHERE FullName = ?
```

and the value

```
' OR '1'='1' --
```

as a separate data message. The value is compared against `FullName` literally and matches no row, so the result is 0 rows. The two cases are fundamentally different: in the vulnerable version the input becomes part of the SQL text; in the safe version the input is data that the server never interprets as SQL.

The method was then deleted and replaced by the `PreparedStatement` version; the same payloads return 0 results.

5.2 5.2 Task 3.2 — Calling `EnrollStudent` via `CallableStatement`

`EnrollService.enrollStudentFromJava` calls the procedure with an OUT parameter:

```
CallableStatement cs = conn.prepareCall("CALL EnrollStudent(?, ?, ?)");
cs.setInt(1, userID);
cs.setInt(2, courseID);
cs.registerOutParameter(3, java.sql.Types.VARCHAR);
cs.execute();
return cs.getString(3);
```

Test results (`evidence/enroll_and_report.txt`):

```
Enrolled successfully (EnrollID=12)
ERROR: already enrolled in this course
ERROR: instructors cannot enroll in courses
```

`registerOutParameter` declares the SQL type of the OUT parameter before `execute()`, so the driver allocates a slot to receive the value the procedure sets. If it is omitted, `cs.getString(3)` throws `SQLException` ("Parameter not registered as output parameter") because the driver does not know that position 3 is an output.

A UI panel with UserID/CourseID fields and an Enroll button was added to `MainFrame` (the search/enrol panels); the button calls this method and shows the returned message in a dialog.

5.3 5.3 Task 3.3 — `GetCourseReport` and multiple result sets

`EnrollService.getCourseReportFromJava` calls `CALL GetCourseReport(?)` and reads the first `ResultSet`:

```
Course Report for CourseID 1:
 Hoang Van Em         | 2024-09-09   | N/A
 Le Van Cuong         | 2024-09-07   | 5.0
 Nguyen Van An        | 2024-09-05   | 8.5
 Tran Thi Bich        | 2024-09-05   | 9.0
```

`cs.execute()` returns `true` when the first result is a `ResultSet` and `false` when it is an update count. A procedure that runs both `UPDATE` and `SELECT` produces multiple results; the Java code must call `cs.getMoreResults()` (and `cs.getUpdateCount()`) in a loop to consume them all. A stored procedure returns multiple result sets when it contains several `SELECT` statements, e.g. a report procedure that returns a summary row set followed by a detail row set. The handling loop is:

```
boolean hasResult = cs.execute();
while (true) {
    if (hasResult) {
        try (ResultSet rs = cs.getResultSet()) { /* read rows */ }
    } else if (cs.getUpdateCount() == -1) {
        break; // no more results
    }
    hasResult = cs.getMoreResults();
}
```

---

6 6. Part 8 — Thinking questions

Q1. Manual sanitisation vs `PreparedStatement`. Escaping quotes by hand is always inferior because the set of dangerous characters depends on the database, the column type, the connection charset, and the SQL context (string literal vs identifier vs numeric). One edge case where manual sanitisation fails: an input containing a backslash in a MySQL string literal with `NO_BACKSLASH_ESCAPES` disabled changes how the following quote is interpreted; another is a multi-byte character whose trailing byte is `0x27` (a quote) in a charset such as GBK, which bypasses `addslashes`-style escaping. `PreparedStatement` avoids all of this by never putting the value into SQL text.

Q2. HikariCP and connection pools. HikariCP is a high-performance JDBC connection pool. With a pool, `getConnection()` no longer opens a socket: it borrows a pre-opened connection from the pool (waiting if none is free). `conn.close()` no longer closes the socket: it returns the connection to the pool. The application code keeps the same try-with- resources shape; only the `DataSource` configuration changes.

Q3. DAO pattern. A separate DAO layer keeps SQL out of the UI event handler, so the UI stays testable and the SQL stays in one place. If EduConnect migrates from MySQL to PostgreSQL, the SQL dialect (date functions, `LIMIT`, identifiers) changes inside the DAO, while the method signatures and the UI stay the same.

Q4. Stored procedure vs JDBC ownership. The lab splits validation: enrolment validation lives in the stored procedure, student name lookup in Java. This is not fully consistent. With a team of five (half DB, half Java), enrolment validation should live in the database, because it must be atomic with the insert and must hold under concurrent calls from any client. The risk is that the DB team changes the procedure's contract (message text, parameters) without telling the Java team, breaking the caller. Mitigation: version the procedure, document the contract, and add integration tests that call it.

Q5. `ResultSet` streaming. `ps.setFetchSize(Integer.MIN_VALUE)` in MySQL Connector/J switches the driver to row-by-row streaming instead of loading the whole result set into memory. The new constraint is that the `ResultSet` (and its `Statement` / `Connection`) must stay open and be consumed sequentially; you cannot issue another query on the same connection until streaming finishes, and you cannot scroll backwards. Choose streaming for very large result sets (reports, exports); choose loading into a `List` for small interactive results.

Q6. Concurrent enrollment. The race happens because the check and the insert are not atomic. The database mechanism that prevents it is a transaction with row-level locking: `SELECT ... FOR UPDATE` on the course row inside a transaction serialises the check-and-insert. Alternatively, a unique constraint on `(UserID, CourseID)` plus an insert that fails on duplicate, or an atomic `INSERT ... SELECT` guarded by the course's capacity. The stored procedure should be rewritten to run in a transaction with `SELECT CurrentCount FROM Courses WHERE CourseID = ? FOR UPDATE`, check capacity, then insert, committing only if the check passes.

7 7. Part 7 checklist

- [x] NetBeans project created with correct structure (Maven project instead, JDK 25)
- [x] MySQL Connector/J JAR added to Libraries (pom.xml dependency)
- [x] `ConnectionTest` runs successfully and prints MySQL version
- [x] Task 1.1: at least 6 issues identified in `BrokenQueryRunner`
- [x] Task 1.2: `SafeQueryRunner` with `PreparedStatement`, try-with-resources, column names
- [x] Task 1.2: normal input returns 1, injection attempt returns 0
- [x] Task 1.2: protocol-level explanation written
- [x] Task 1.3: connection leak demonstrated; MySQL error 1040 documented
- [x] Task 1.3: max-connections math calculated
- [x] Task 2.1: `MainFrame` runs; both test queries display in `JTable`
- [x] Task 2.1: button disables during query and re-enables after
- [x] Task 2.1: `SwingWorker` necessity explained
- [x] Task 2.2: parameterised search with `PreparedStatement` and `LIKE`
- [x] Task 2.2: `LIKE` safety explanation written
- [x] Task 2.3: bad `ResultSet` exception observed and documented
- [x] Task 2.3: rewritten method returns `List` instead of `ResultSet`
- [x] Task 3.1: injection payloads tested; SQL drawn out
- [x] Task 3.1: `PreparedStatement` version returns 0 results
- [x] Task 3.2: `EnrollStudent` called via `CallableStatement`; all 3 test cases pass
- [x] Task 3.2: `registerOutParameter` explained
- [x] Task 3.3: `GetCourseReport` called; `ResultSet` read correctly
- [x] No raw `Statement` used except the Task 3.1 demonstration
- [x] All `Connection`, `PreparedStatement`, `ResultSet` use try-with-resources
- [x] No column index access in final code

---

8 8. Reflection

The most surprising part of this lab was how little code separates a safe query from a catastrophic one: a single `+` in a string is the difference between a parameter and an injection. Seeing the payloads return the entire `Users` table made the risk concrete. The connection-leak demonstration was equally sobering — 152 unclosed connections took down the server in under a second. The two habits I will carry forward are: never concatenate user input into SQL, and always close resources with try-with-resources, even on the happy path.

---

9 Appendix A — Contents of the submission bundle

File	Purpose
`PDMLab6_ITCSIU25055_LeThanhTrung.pdf`	this report
`PDMLab6_ITCSIU25055.docx`	editable copy
`project/pom.xml`	Maven build with Connector/J 8.4.0
`project/src/main/java/educonnect/*.java`	all Java classes
`src/00_db_objects.sql`	view + stored procedures
`evidence/*.txt`	captured output of every demonstration
`README.md`	how to build and run